

BrokerVerse OOTB

Insurance broking platform by iorta TechNXT

Solution Architecture · Document 11 of 11

Monitoring

Product	BrokerVerse OOTB
Document	11 Monitoring
Version	1.1
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse OOTB Solution Architecture: 11 Monitoring
Document ID	BV-SA-11
Version	1.1
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, deploy/, docker-compose.yml, .github/workflows, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue
1.1	29 September 2026	iorta TechNXT	Updated to the current system: placement journey, co-insurance, posting rules, period end and BIR tax, bank reconciliation, document numbering, commission rate matrix, single PDF engine, uploads and go-live imports, housekeeping, business time zone, broker roles, deployment on the BrokerVerse URL

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

- 1 Introduction..... 4
 - 1.1 Purpose and scope..... 4
- 2 Telemetry produced by the application..... 6
 - 2.1 Logs..... 6
 - 2.2 Health and version endpoints..... 6
 - 2.3 Operational data in the database..... 6
- 3 Metrics to collect..... 7
 - 3.1 Application metrics exporter..... 7
- 4 Dashboards and alarms..... 9
 - 4.1 Dashboard "BrokerVerse production"..... 9
 - 4.2 Alarms..... 9
- 5 Audit trail review..... 11
- 6 Synthetic checks..... 12
- 7 On-call runbook..... 13
- 8 Related documents..... 14
- 9 Glossary..... 15

1 Introduction

1.1 Purpose and scope

This document describes how BrokerVerse OOTB is monitored: the telemetry the application produces (logs, health and version endpoints, operational tables), the metrics to collect, the recommended CloudWatch dashboards and alarms with thresholds, the review of the audit trail, synthetic checks, and the on-call runbook entries for the most likely alerts.

Recommended / to be confirmed by the business and DevOps: The code produces the telemetry described in chapter 2. The deployment package (`deploy/`) contains no monitoring configuration (no dashboards, alarms, log shipping or metrics exporter). Metrics, thresholds, dashboards and runbooks in chapters 3 to 7 are recommendations.

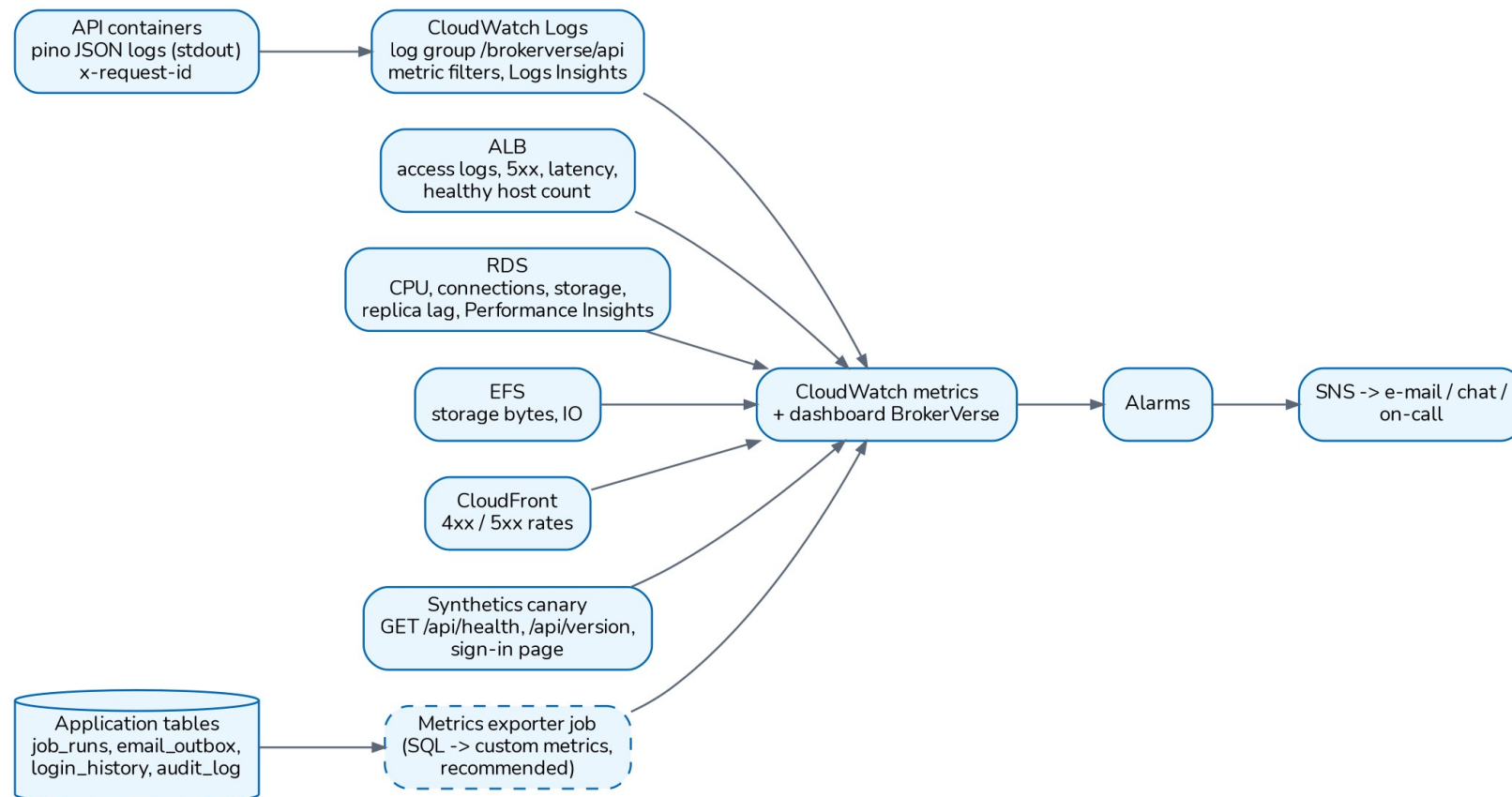


Figure 1. Monitoring and alerting (recommended)

2 Telemetry produced by the application

2.1 Logs

Aspect	Implementation
Format	pino JSON lines on standard output (one object per line: <code>level</code> , <code>time</code> , <code>pid</code> , <code>hostname</code> , <code>msg</code> , plus context). On AWS the container log driver (awslogs) ships them to CloudWatch Logs (recommended log group <code>/brokerverse/api</code>).
Access log	pino-http logs one line per request with <code>req</code> (id, method, url, query, headers), <code>res</code> (status code, headers) and <code>responseTime</code> in milliseconds. Health checks (<code>/api/health</code> , <code>/api/health/live</code>) are not logged.
Correlation	Every request gets <code>x-request-id</code> (the caller's value when well formed, otherwise a new UUID); it is the log <code>req.id</code> , returned as a response header and included in every error body, so a user can quote it and support can find the log lines.
Redaction	<code>authorization</code> , <code>cookie</code> , <code>x-api-key</code> , <code>set-cookie</code> headers and <code>password</code> , <code>newPassword</code> , <code>currentPassword</code> , <code>refreshToken</code> , <code>code</code> body fields are replaced by <code>[redacted]</code> ; <code>token</code> , <code>sig</code> , <code>code</code> , <code>access_token</code> , <code>refresh_token</code> , <code>password</code> query parameters are masked in logged URLs.
Errors	5xx: full error (message, stack, database error) logged at <code>error</code> level with the request id; the client receives a generic message. 4xx: the message is returned to the client.
Levels	<code>LOG_LEVEL</code> (default <code>info</code>); start-up (applied <code><migration></code> , <code>seed complete</code> , BrokerVerse API listening on <code>:8000</code> , <code>scheduler: N job(s) scheduled (time zone Asia/Manila)</code>), shutdown and fatal configuration errors are logged. A failed job run is logged at <code>error</code> with <code>job</code> and <code>runId</code> (<code>scheduled job <code> failed</code>); job skips are logged at <code>debug</code> .
Front end	Diagnostics go through <code>utility/logger.js</code> , which is silent in production builds; browser errors are not collected centrally.

2.2 Health and version endpoints

GET `/api/health` (readiness with database check and pending migrations), GET `/api/health/live` (liveness) and GET `/api/version` (version, commit from `GIT_COMMIT`, start time, uptime, Node version, database latency, pending migrations); details in document 07. Set `GIT_COMMIT` in the image build so the running release can be identified.

2.3 Operational data in the database

Table	Monitoring use
<code>job_runs</code> , <code>scheduled_jobs</code>	Every scheduled or manual job run with status (<code>success</code> / <code>failed</code>), start and end time, output or error; <code>scheduled_jobs.last_status</code> per job. Visible in Master > Schedules > History. The housekeeping output lists the rows deleted per table.
<code>email_outbox</code>	Queued, sent and failed e-mails with attempts and last error.
<code>login_history</code>	Every sign-in attempt with success flag and reason (<code>bad-password</code> , <code>locked</code> , <code>unknown-user</code> , <code>inactive</code> , <code>2fa-required</code> , <code>refresh-token-reuse</code> ...), IP and user agent.
<code>users</code>	<code>status = 'locked'</code> , <code>failed_logins</code> , <code>last_login_at</code> , <code>password_changed_at</code> , <code>totp_enabled</code> .
<code>audit_log</code>	All business and administrative changes (chapter 5).
<code>period_close_runs</code> , <code>accounting_periods</code> , <code>bank_reconciliations</code>	Month-end close status per period and the last reconciled month per bank account (business monitoring by the Accounting Manager).
<code>generated_reports</code>	Report generation status and errors.
<code>documents</code>	Stored files and sizes (disk usage by category).

3 Metrics to collect

Area	Metric	Source
Availability	Healthy host count; synthetic check success	ALB HealthyHostCount; CloudWatch Synthetics
Traffic and errors	Requests per minute; HTTP 5xx and 4xx rate; 429 rate (rate limiting)	ALB RequestCount, HTTPCode_Target_5XX_Count, HTTPCode_Target_4XX_Count; log metric filter on <code>res.statusCode = 429</code>
Latency	p50 / p95 / p99 target response time; slow requests (> 3 s)	ALB TargetResponseTime; log metric filter on <code>responseTime > 3000</code>
API compute	CPU and memory utilisation per container; running container count; restarts	Container service metrics (for example ECS Container Insights)
Database	CPU, freeable memory, DatabaseConnections, read / write IOPS and latency, free storage, replica lag, deadlocks, top SQL	RDS metrics, Performance Insights
Files	EFS StorageBytes, PercentIOLimit, throughput; growth per month	EFS metrics
Scheduled jobs	Failed runs per job in the last 24 hours; hours since last successful run per job; run duration	Log metric filter on <code>scheduled_job * failed</code> ; SQL on <code>job_runs</code> (exporter below)
E-mail	Queued messages older than 15 minutes; failed messages per day	SQL on <code>email_outbox</code>
Security	Failed sign-ins per 5 minutes; locked accounts; refresh-token reuse events; 401 / 403 rate; administrator changes to users and roles	SQL on <code>login_history</code> , <code>users</code> , <code>audit_log</code> ; ALB 4xx
Front end	CloudFront 4xx / 5xx error rate; cache hit rate	CloudFront metrics
Certificates and backups	ACM certificate days to expiry; AWS Backup job failures; latest restorable time of RDS	ACM, AWS Backup, RDS

3.1 Application metrics exporter

Recommended / to be confirmed by the business and DevOps: Publish the database-derived metrics with a small scheduled task (for example an EventBridge-scheduled Lambda or container task every 5 minutes with read-only database access, or a new scheduled job in the API using `PutMetricData` / CloudWatch embedded metric format in the log) running queries such as:

Metric (namespace BrokerVerse)	Query
JobFailures24h (per job)	<code>SELECT j.code, count(*) FROM job_runs r JOIN scheduled_jobs j ON j.id = r.job_id WHERE r.status = 'failed' AND r.started_at > now() - interval '24 hours' GROUP BY j.code</code>
HoursSinceSuccess (per job)	<code>SELECT code, extract(epoch FROM now() - max(r.finished_at)) / 3600 FROM scheduled_jobs j LEFT JOIN job_runs r ON r.job_id = j.id AND r.status = 'success' WHERE j.enabled GROUP BY code</code>
EmailQueuedOld / EmailFailed24h	<code>SELECT count(*) FROM email_outbox WHERE status = 'queued' AND created_at < now() - interval '15 minutes'; ... WHERE status = 'failed' AND created_at > now() - interval '24 hours'</code>

Metric (namespace BrokerVerse)	Query
LoginFailures5m / LockedUsers	SELECT count(*) FROM login_history WHERE NOT success AND reason NOT IN ('2fa-required', '2fa-setup-required') AND at > now() - interval '5 minutes';SELECT count(*) FROM users WHERE status = 'locked'
RefreshTokenReuse	SELECT count(*) FROM login_history WHERE reason = 'refresh-token-reuse' AND at > now() - interval '1 hour'
UploadsBytes	SELECT sum(size_bytes) FROM documents (compare with EFS StorageBytes)

4 Dashboards and alarms

4.1 Dashboard "BrokerVerse production"

1. Availability: synthetic check status, ALB healthy hosts, 5xx rate, requests per minute.
2. Latency: ALB target response time p50 / p95 / p99; slow requests.
3. API containers: CPU, memory, container count, deployments.
4. Database: CPU, connections, free storage, IOPS, latency, replica lag; Performance Insights top SQL link.
5. Business operations: job failures and hours since success per job, e-mail queue and failures, generated reports.
6. Security: failed sign-ins, locked users, refresh-token reuse, 401 / 403 / 429 rates.
7. Storage: EFS size and monthly growth, RDS storage growth.

4.2 Alarms

Alarm (Recommended / to be confirmed by the business and DevOps)	Threshold	Severity	Notify
Synthetic health check failing	2 consecutive failures (checks every 5 min)	P1	On-call (SNS: phone / chat), DevOps
ALB healthy hosts below 1	1 minute	P1	On-call
HTTP 5xx rate	> 2 % of requests for 5 min, or > 20 5xx in 5 min	P1	On-call
p95 target response time	> 2 s for 10 min	P2	DevOps
API container CPU	> 80 % for 15 min (auto scaling should act first)	P2	DevOps
API container memory	> 85 % for 10 min	P2	DevOps
RDS CPU	> 80 % for 15 min	P2	DevOps
RDS free storage	< 20 %	P2	DevOps
RDS connections	> 80 % of <code>max_connections</code>	P2	DevOps
RDS failover / reboot event	any	P2	DevOps (information)
Scheduled job failed	any failure of <code>policy-expiry</code> , <code>receivable-ageing</code> , <code>renewal-pipeline</code> , <code>renewal-notices</code> , <code>collection-reminders</code> , <code>daily-reports</code> , <code>housekeeping</code> and, once enabled, the <code>period-end</code> and <code>bank-matching</code> jobs; 3 consecutive failures of <code>email-outbox</code> or <code>renewal-queue</code>	P2	DevOps, business application support
Job not run	Hours since success > 26 for daily jobs; > 0.5 for <code>email-outbox</code>	P2	DevOps
E-mail backlog	Queued messages older than 15 min > 0 for 30 min (when e-mail is enabled); failed > 10 per day	P3	Application support
Failed sign-ins	> 50 in 5 min (possible credential attack)	P2	Security
Refresh-token reuse	any	P3	Security (review the user's sessions)
Locked accounts	> 5 new locks per hour	P3	Security, service desk

Alarm (Recommended / to be confirmed by the business and DevOps)	Threshold	Severity	Notify
EFS growth	> 20 GB in a day or > 80 % of the budgeted size	P3	DevOps
Backup job failed / RDS latest restorable time older than 15 min	any	P2	DevOps
ACM certificate expiry	< 30 days	P3	DevOps
CloudFront 5xx rate	> 1 % for 10 min	P2	DevOps

Severity: P1 immediate response (24 x 7), P2 same business day, P3 next business day (to be aligned with the support contract).

5 Audit trail review

Review	Recommendation
Access reviews	Monthly: users with the System Administrator role (<code>system-admin</code>) and the Accounting Manager role, new users, role and permission changes (<code>audit_log</code> entities for users and roles), dormant accounts (<code>last_login_at</code> older than 90 days).
Sensitive changes	Weekly: changes to settings (<code>app_settings</code> : tax rates, account determination, security policy), posting rules, document number series, tax codes, scheduled jobs, chart of accounts, the commission rate matrix, the Company master (letterhead), direct-bill mode changes, journal reversals and corrections, cancelled receipts, period reopenings.
Maker-checker exceptions	Monthly: approvals by delegation (<code>remittance_delegations</code>), returned or rejected settlements, reversed commission lines, rejected or reopened close runs and bank reconciliations, postings into soft-closed periods.
Security events	Weekly: failed sign-ins by IP, locked accounts, refresh-token reuse, two-factor disabled by administrators.
Tooling	Master > Audit Trail filters by entity, entity id, username and a from / to date range (newest 200, at most 1,000 rows), backed by indexes on (<code>entity</code> , <code>entity_id</code>), (<code>at</code>) and (<code>username</code>). For periodic reviews use SQL on <code>audit_log</code> by date, or export the audit log to CloudWatch Logs or a SIEM.
Retention	Document 10.

6 Synthetic checks

Check	Definition (Recommended)
API readiness	Every 5 minutes from outside AWS or CloudWatch Synthetics: GET <code>https://<brokerverse-url>/api/health</code> returns 200 and <code>"ready":true</code> within 2 s
Version	Every 15 minutes: GET <code>/api/version</code> ; alert when <code>pendingMigrations</code> is not 0 or the commit differs from the expected release after a deployment
Front end	Every 5 minutes: the sign-in page loads (HTTP 200, the SPA bundle referenced by <code>index.html</code> loads)
Sign-in journey	Every 15 minutes with a dedicated monitoring user without business permissions (only <code>profile</code>): POST <code>/api/auth/login</code> , GET <code>/api/auth/profile</code> , POST <code>/api/auth/logout</code> ; exclude the monitoring user from lockout investigations
Signed file link	Daily: open a known test document through a signed link returned by the API; confirm that the same URL without <code>sig</code> is refused (security regression)

7 On-call runbook

Alert	First actions
Health check failing / no healthy hosts	Check the container service events and the reasons containers stopped; read the last log lines of a container (start-up errors such as <code>Refusing to start in production: ... or migration ... failed</code>). If the database is unreachable (<code>database.reachable: false</code> in <code>/api/health</code>), check RDS status and events (failover in progress?) and security groups. If a migration failed, roll back to the previous image (document 09, release rollback) and investigate on UAT.
5xx spike	Find the request ids of failing requests in CloudWatch Logs Insights (<code>filter res.statusCode >= 500 stats count() by req.url</code>), read the error lines with the same <code>req.id</code> ; check recent deployments and configuration changes (<code>audit_log</code> entity settings); roll back if caused by the release.
High latency / CPU	Check whether large report generations or imports are running (log lines for <code>/reports/*/generate</code> , import endpoints); check RDS Performance Insights for slow SQL and missing indexes; add API instances.
Database storage / connections	Increase storage (autoscaling); check that the <code>housekeeping</code> job succeeds and look for growth in <code>audit_log</code> and the transaction tables (document 10); check the connection count against instances x 10.
Scheduled job failed	Master > Schedules > History shows the error; fix the cause (data, configuration, SMTP) and use Run now. Daily jobs are safe to re-run (they are idempotent by design: expiry and ageing recompute, notices check for duplicates).
E-mail backlog / failures	Check <code>notification.email_enabled</code> , <code>SMTP_URL</code> secret, the Office 365 service status (Authenticated SMTP on for the mailbox) and the <code>error</code> column of <code>email_outbox</code> ; messages with 5 failed attempts stay <code>failed</code> and must be re-queued (<code>UPDATE email_outbox SET status = 'queued', attempts = 0 WHERE ...</code>) after the cause is fixed.
Failed sign-in spike / token reuse	Identify source IPs in <code>login_history</code> ; block at WAF if it is an attack; for token reuse, contact the user, revoke sessions (User Management) and check the device.
Locked administrator account	Another System Administrator sets the account back to active in User Management; if no administrator can sign in, follow the break-glass procedure (database update of <code>users.status</code> and <code>failed_logins</code> under change control, audited).
Backup failure	Re-run the backup job; confirm RDS latest restorable time; escalate if PITR is not available.

Runbook entries should be kept in the operations wiki with links to the dashboard, log group, container service, database instance and this document set; each alarm description should link to its runbook entry.

8 Related documents

This document is one of eleven that together form the BrokerVerse OOTB Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring (this document)	11_BrokerVerse_Monitoring.pdf

9 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Account determination	Settings accounting.account.<role> that give the GL account of each account role (cash in bank, premium receivable, due to insurer, commission income ...) used by the posting rules.
ACM	AWS Certificate Manager: issues and renews the TLS certificates used by the load balancer and CloudFront.
ALB	Application Load Balancer (AWS): distributes HTTPS requests over the API containers and checks their health.
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
Audit log / audit trail	Table audit_log: who changed which record, when, from which IP, with the before and after values.
CloudFront	AWS content delivery network that serves the single-page application and can route /api to the backend.
CloudWatch	AWS monitoring service: logs, metrics, dashboards and alarms.
Company master	Master record of the broker company; the primary company is the letterhead printed on every document and report.
ECS / Fargate	Elastic Container Service (AWS) and its serverless compute mode for running the API containers.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
Housekeeping	Daily job that deletes operational rows (job runs, sent e-mails, sign-in history, expired tokens, read notifications) after the retention days in System Settings.
Maker-checker	Control in which a record created or submitted by one user must be approved by a different user.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Month-end close	Run (MEC-) that posts the accruals, recurring journals and valuations of a period, checks the checklist and, once approved, closes the period.
Office 365	Microsoft 365 mail service; BrokerVerse sends e-mail through smtp.office365.com with the mailbox in SMTP_URL.
p50 / p95 / p99	Latency percentiles: the time within which 50 %, 95 % or 99 % of requests completed.
PITR	Point-in-time recovery: restoring a database to any second inside the backup retention window.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
Seed	Idempotent data load run on every API start: reference data always, sample data only when SEED_SAMPLE_DATA is on.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in SMTP_URL.
SNS	Amazon Simple Notification Service: delivers alarm notifications.
SPA	Single-page application: the React front end served as static files.
UAT	User acceptance testing environment.
WAF	Web Application Firewall.

